

Aegis AML — Database Schema

Version: 1.0 — MVP **Engine:** PostgreSQL 15 **ORM:** SQLAlchemy 2.0 with Alembic migrations **Last Updated:** 2026-06-10

Table of contents

- [Design Principles](#)
 - [Entity Relationship Summary](#)
 - [Table Definitions](#)
 - [1. tenants](#)
 - [2. users](#)
 - [3. ingestion_schemas](#)
 - [4. webhook_configs](#)
 - [5. llm_configs](#)
 - [6. alerts](#)
 - [7. pii_maps](#)
 - [8. compliance_matches](#)
 - [9. sar_drafts](#)
 - [10. webhook_deliveries](#)
 - [11. webhook_sink_events](#)
 - [12. audit_logs](#)
 - [13. api_logs](#)
 - [Seed Data](#)
 - [Super Admin User](#)
 - [Demo Tenant \(Pre-approved for demo sessions\)](#)
 - [Preset Schema Templates \(Hardcoded in Backend\)](#)
 - [Migration Strategy](#)
-

Design Principles

1. **Row-level multi-tenancy.** Every data table has a `tenant_id` FK. No cross-tenant queries are possible without explicitly providing the wrong `tenant_id` (enforced at service layer).
 2. **Soft deletes where data has legal value.** Alerts, SARs, and audit logs are never hard-deleted in production.
 3. **JSONB for variable-structure data.** Schema maps, raw payloads, and compliance rule results are stored as JSONB — avoids premature column proliferation.
 4. **Append-only audit log.** `audit_logs` has no UPDATE or DELETE operations. Ever.
 5. **Timestamps on everything.** Every table has `created_at`. Tables tracking state changes also have `updated_at`.
-

Entity Relationship Summary

```

tenants
├── users (1:many)
├── ingestion_schemas (1:many)
├── webhook_configs (1:1)
├── llm_configs (1:1)
├── alerts (1:many)
│   ├── pii_maps (1:1)
│   ├── compliance_matches (1:many)
│   └── sar_drafts (1:1)
│       └── webhook_deliveries (1:many)
├── webhook_sink_events (1:many)
├── audit_logs (1:many)
└── api_logs (1:many)

```

Table Definitions

1. tenants

The root entity. Each B2B client (fintech / broker) is one tenant.

```

CREATE TABLE tenants (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name              VARCHAR(255) NOT NULL,
    slug              VARCHAR(100) UNIQUE NOT NULL,           -- URL-safe identifier
    e.g. "payu-india"
    company_type      VARCHAR(50) NOT NULL,                   -- FINTECH | BROKER |
    NBFC | PAYMENT_CO | OTHER
    cin               VARCHAR(50),                             -- Corporate Identity
    Number (India)
    sebi_reg_no       VARCHAR(50),                             -- SEBI Registration
    Number (for brokers)
    website           VARCHAR(255),

    -- Status machine
    status            VARCHAR(30) NOT NULL DEFAULT 'PENDING_VERIFICATION',
                                -- PENDING_VERIFICATION | ACTIVE | REJECTED | SUSPENDED
    rejection_reason  TEXT,                                     -- Set when status =
    REJECTED

    -- API credentials (generated on approval)
    api_key_hash      VARCHAR(255),                             -- bcrypt hash of the
    actual API key
    api_key_prefix    VARCHAR(12),                             -- First 8 chars for
    display e.g. "sk-ae-a1b2"
    tenant_id_public  VARCHAR(20) UNIQUE,                     -- Short public tenant
    ID e.g. "TEN-0001"

    -- Metadata

```

```

        approved_at      TIMESTAMPTZ,
        approved_by      UUID,                                -- FK to users (super
admin)
        suspended_at     TIMESTAMPTZ,
        suspended_by      UUID,

        created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );

    CREATE INDEX idx_tenants_status ON tenants(status);
    CREATE INDEX idx_tenants_api_key_prefix ON tenants(api_key_prefix);

```

Notes:

- The actual plaintext API key is returned ONCE at approval and never stored. Only the bcrypt hash is stored.
- `api_key_prefix` is used to identify which tenant an incoming request is from before running the bcrypt check.
- `tenant_id_public` is the `X-Tenant-ID` header value clients use.

2. users

Portal users. For MVP, one user per tenant (the admin). Post-MVP, multiple users with roles.

```

CREATE TABLE users (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id   UUID REFERENCES tenants(id) ON DELETE CASCADE,
                -- NULL for SUPER_ADMIN users (they belong to no tenant)

    email       VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,                                -- bcrypt
    full_name    VARCHAR(255) NOT NULL,
    designation   VARCHAR(100),
    phone        VARCHAR(20),

    role         VARCHAR(30) NOT NULL,
                -- SUPER_ADMIN | TENANT_ADMIN | COMPLIANCE_OFFICER

    is_active    BOOLEAN NOT NULL DEFAULT TRUE,
    last_login_at TIMESTAMPTZ,

    -- JWT refresh token tracking
    refresh_token_hash VARCHAR(255),
    refresh_token_exp   TIMESTAMPTZ,

    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```
CREATE INDEX idx_users_tenant_id ON users(tenant_id);
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);
```

3. ingestion_schemas

Defines how a tenant's raw JSON alert payload maps to Aegis standard fields.

```
CREATE TABLE ingestion_schemas (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id         UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,

  name              VARCHAR(255) NOT NULL,                -- e.g. "Standard
  Fintech Alert"
  template_key      VARCHAR(50),                          -- STANDARD_FINTECH |
  SEBI_BROKER | PAYMENT_GW | CUSTOM
  is_active         BOOLEAN NOT NULL DEFAULT TRUE,

  -- The actual mapping: Aegis standard field → tenant's JSON key path
  -- Example: {"customer_name": "user.full_name", "account_id":
  "account.number", ...}
  field_map         JSONB NOT NULL DEFAULT '{}',

  -- Which fields contain PII and should be masked
  -- Example: ["customer_name", "account_id", "counterparty_account",
  "ip_address"]
  pii_fields        JSONB NOT NULL DEFAULT '[]',

  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_schemas_tenant_id ON ingestion_schemas(tenant_id);
CREATE INDEX idx_schemas_active ON ingestion_schemas(tenant_id, is_active);
```

Aegis Standard Fields (the target of all schema mappings):

Standard Field	Description	PII?
customer_name	Full legal name of the subject	YES
customer_id	Internal customer identifier	YES
account_id	Source account number	YES
transaction_id	Unique transaction reference	NO
transaction_amount	Amount in base currency unit	NO
transaction_currency	ISO currency code	NO

Standard Field	Description	PII?
transaction_type	e.g. TRANSFER, WITHDRAWAL, PURCHASE	NO
transaction_direction	CREDIT / DEBIT	NO
transaction_timestamp	ISO 8601 datetime	NO
counterparty_account	Destination/source account	YES
counterparty_name	Destination/source name	YES
counterparty_institution	Bank/NBFC name	NO
ip_address	Client IP at time of transaction	YES
device_id	Device fingerprint	YES
risk_score	Numeric risk score (0–100)	NO
alert_reason	TMS-provided reason text	NO
geo_country	Country of origin	NO

4. webhook_configs

One webhook configuration per tenant.

```
CREATE TABLE webhook_configs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL UNIQUE REFERENCES tenants(id) ON DELETE CASCADE,

  callback_url VARCHAR(500),           -- NULL if using
  internal_sink
  use_internal_sink BOOLEAN NOT NULL DEFAULT FALSE, -- TRUE = built-in
  test_receiver

  -- HMAC signing secret (generated on save, revealed once)
  secret_hash  VARCHAR(255),           -- bcrypt hash
  secret_prefix VARCHAR(12),           -- First chars for
  display

  is_active    BOOLEAN NOT NULL DEFAULT TRUE,
  last_tested_at TIMESTAMPTZ,
  last_test_status VARCHAR(20),       -- SUCCESS | FAILED |
  NULL

  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

5. llm_configs

LLM provider settings per tenant. MVP only supports Groq (SaaS Managed).

```
CREATE TABLE llm_configs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL UNIQUE REFERENCES tenants(id) ON DELETE CASCADE,

  provider     VARCHAR(30) NOT NULL DEFAULT 'GROQ',      -- GROQ | PRIVATE
  (future)
  model_name   VARCHAR(100) NOT NULL DEFAULT 'llama-3.3-70b-versatile',
  sar_template_style VARCHAR(20) NOT NULL DEFAULT 'BOTH', -- NARRATIVE |
  STRUCTURED | BOTH

  -- For PRIVATE provider (future)
  private_base_url  VARCHAR(500),
  private_token_hash VARCHAR(255),                -- Encrypted, not
  bcrypt

  -- Token usage tracking
  total_tokens_used  BIGINT NOT NULL DEFAULT 0,
  total_requests     INTEGER NOT NULL DEFAULT 0,

  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

6. alerts

Each flagged transaction alert ingested from a tenant's TMS.

```
CREATE TABLE alerts (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id) ON DELETE RESTRICT,
  schema_id   UUID REFERENCES ingestion_schemas(id),

  -- Status machine
  status       VARCHAR(30) NOT NULL DEFAULT 'PENDING_INGESTION',
               -- PENDING_INGESTION | PROCESSING | PENDING_REVIEW
               -- | APPROVED | REJECTED | DELIVERED | DELIVERY_FAILED

  -- Raw and processed payloads
  -- raw/normalized carry REAL PII → Fernet-encrypted at rest by the app layer
  -- (EncryptedJSONB, stored as {"__enc__": "<token>"} inside the JSONB; legacy
  -- plaintext rows still decode). masked stays plaintext – tokens only, no PII.
  raw_payload  JSONB NOT NULL,                      -- Exact payload
  received from TMS (ENCRYPTED at rest)
  normalized_payload JSONB,                          -- After schema
  mapping, before masking (ENCRYPTED at rest)
  masked_payload  JSONB,                             -- After PII masking
  (tokens, not real values)
```

```

-- Extracted standard fields (denormalized for easy querying)
transaction_id VARCHAR(255),
transaction_amount NUMERIC(20, 4),
transaction_currency VARCHAR(10) DEFAULT 'INR',
transaction_type VARCHAR(50),
transaction_timestamp TIMESTAMPTZ,
risk_score INTEGER, -- 0-100

-- Processing metadata
processing_started_at TIMESTAMPTZ,
processing_completed_at TIMESTAMPTZ,
processing_error TEXT, -- Set if pipeline
failed

-- Review metadata
reviewed_by UUID REFERENCES users(id),
reviewed_at TIMESTAMPTZ,
rejection_reason TEXT,

-- Soft delete
is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
deleted_at TIMESTAMPTZ,

-- Source tracking
source VARCHAR(20) NOT NULL DEFAULT 'API', -- API | SIMULATOR
ingested_from_ip VARCHAR(50),

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_alerts_tenant_id ON alerts(tenant_id);
CREATE INDEX idx_alerts_status ON alerts(tenant_id, status);
CREATE INDEX idx_alerts_created_at ON alerts(tenant_id, created_at DESC);
CREATE INDEX idx_alerts_transaction_id ON alerts(tenant_id, transaction_id);

```

7. pii_maps

Stores the tokenization mapping for a specific alert. One row per alert.

```

CREATE TABLE pii_maps (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  alert_id UUID NOT NULL UNIQUE REFERENCES alerts(id) ON DELETE CASCADE,
  tenant_id UUID NOT NULL REFERENCES tenants(id),

  -- The token ↔ real value mapping – Fernet-ENCRYPTED at rest by the app layer
  -- (EncryptedJSONB: stored as {"__enc__": "<token>"}; a DB dump exposes only
  ciphertext)
  -- Decrypted example: {"USR_a1b2c3d4": "Rajesh Kumar", "ACC_e5f6g7h8": "HDFC-

```

```

00123456"}
    token_map          JSONB NOT NULL,

    -- Audit
    masked_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    rehydrated_at       TIMESTAMPTZ,                                -- Set when
rehydration happens on approval

    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_pii_maps_tenant_id ON pii_maps(tenant_id);

```

Token Format:

- User tokens: `USR_<8 random hex chars>` e.g. `USR_a1b2c3d4`
- Account tokens: `ACC_<8 hex>` e.g. `ACC_e5f6a7b8`
- Transaction tokens: `TXN_<8 hex>`
- IP tokens: `IP_<8 hex>`
- Device tokens: `DEV_<8 hex>`

8. `compliance_matches`

Records which AML typology rules fired for an alert, and the evidence.

```

CREATE TABLE compliance_matches (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    alert_id          UUID NOT NULL REFERENCES alerts(id) ON DELETE CASCADE,
    tenant_id         UUID NOT NULL REFERENCES tenants(id),

    rule_id           VARCHAR(50) NOT NULL,
                        -- STRUCTURING | RAPID_MOVEMENT | ROUND_NUMBER |
DORMANT_ACTIVATION   -- | HIGH_RISK_TYPE | VELOCITY | COUNTERPARTY_RISK |
RISK_SCORE_THRESHOLD

    rule_name         VARCHAR(255) NOT NULL,                                -- Human-readable name
    triggered         BOOLEAN NOT NULL,                                    -- TRUE = fired, FALSE
= clean
    confidence        VARCHAR(10) NOT NULL DEFAULT 'LOW',                -- HIGH | MEDIUM | LOW

    -- Why this rule fired: specific field values + explanation
    evidence          JSONB NOT NULL DEFAULT '{}',
    -- Example: {"field": "transaction_amount", "value": "990000",
    --           "explanation": "Amount is ₹9.9L, just below ₹10L reporting
threshold"}

    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```



```
CREATE INDEX idx_matches_alert_id ON compliance_matches(alert_id);
CREATE INDEX idx_matches_tenant_rule ON compliance_matches(tenant_id, rule_id,
triggered);
```

9. sar_drafts

The AI-generated SAR draft associated with an alert. One-to-one with alerts.

```
CREATE TABLE sar_drafts (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  alert_id          UUID NOT NULL UNIQUE REFERENCES alerts(id) ON DELETE RESTRICT,
  tenant_id         UUID NOT NULL REFERENCES tenants(id),

  -- Draft content
  draft_text        TEXT NOT NULL,                                -- LLM-generated
  narrative (masked PII)
  draft_structured  JSONB,                                       -- Structured fields
  if template_style = STRUCTURED or BOTH

  -- Edit tracking
  officer_edit_count INTEGER NOT NULL DEFAULT 0,
  last_edited_by    UUID REFERENCES users(id),
  last_edited_at    TIMESTAMPTZ,

  -- Final approved version (populated on finalization – auto-approve or
  officer)
  -- Both carry REAL PII → Fernet-ENCRYPTED at rest by the app layer
  (EncryptedText,
  -- stored with an "enc::v1::" prefix; legacy plaintext rows still decode)
  approved_text     TEXT,                                         -- Finalized text,
  real PII (ENCRYPTED at rest)
  rehydrated_text   TEXT,                                         -- Rehydrated text,
  real PII (ENCRYPTED at rest)
  pdf_path          VARCHAR(500),                                  -- LEGACY/unused: live
  PDFs are rendered in-memory,
  -- never persisted
  (changed 2026-07-05)
  pdf_generated_at  TIMESTAMPTZ,

  -- LLM metadata
  llm_provider      VARCHAR(30),
  llm_model         VARCHAR(100),
  prompt_tokens     INTEGER,
  completion_tokens INTEGER,
  generation_latency_ms INTEGER,

  -- Groq generation prompt stored for auditability
  prompt_used       TEXT,

  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```

        updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );

    CREATE INDEX idx_sar_drafts_tenant_id ON sar_drafts(tenant_id);
    CREATE INDEX idx_sar_drafts_alert_id ON sar_drafts(alert_id);

```

10. webhook_deliveries

Tracks each delivery attempt for an approved SAR.

```

CREATE TABLE webhook_deliveries (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sar_draft_id      UUID NOT NULL REFERENCES sar_drafts(id) ON DELETE RESTRICT,
    tenant_id         UUID NOT NULL REFERENCES tenants(id),

    destination_url   VARCHAR(500) NOT NULL,                -- The URL we
    attempted to POST to
    is_internal_sink   BOOLEAN NOT NULL DEFAULT FALSE,

    -- Delivery status
    status            VARCHAR(20) NOT NULL DEFAULT 'PENDING',
                                -- PENDING | DELIVERED | FAILED | RETRYING

    -- Attempt tracking
    attempt_number     INTEGER NOT NULL DEFAULT 1,
    max_attempts       INTEGER NOT NULL DEFAULT 3,

    -- HTTP details
    http_status_code   INTEGER,
    response_body       TEXT,
    request_headers     JSONB,
    request_body_hash   VARCHAR(64),                        -- SHA256 of body (not
    storing full body)

    -- HMAC
    hmac_signature     VARCHAR(128),                        -- The X-Aegis-
    Signature we sent

    -- Timing
    attempted_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    delivered_at        TIMESTAMPTZ,
    next_retry_at       TIMESTAMPTZ,

    error_message       TEXT,

    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_deliveries_sar_draft_id ON webhook_deliveries(sar_draft_id);
CREATE INDEX idx_deliveries_tenant_status ON webhook_deliveries(tenant_id,

```

```
status);
CREATE INDEX idx_deliveries_next_retry ON webhook_deliveries(next_retry_at)
WHERE status = 'RETRYING';
```

11. webhook_sink_events

Stores payloads received by the built-in test webhook receiver.

```
CREATE TABLE webhook_sink_events (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,

  -- Received payload – approval events include the goAML STR (REAL PII) →
  -- Fernet-ENCRYPTED at rest by the app layer (EncryptedJSONB). Headers hold no
  PII.
  payload     JSONB NOT NULL,
  headers     JSONB,

  -- Verification
  hmac_valid   BOOLEAN,                                -- Did signature
  verification pass?,

  -- Source tracking
  source_ip    VARCHAR(50),
  delivery_id  UUID REFERENCES webhook_deliveries(id),

  received_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_sink_events_tenant_id ON webhook_sink_events(tenant_id,
received_at DESC);
```

12. audit_logs

Immutable audit trail. No UPDATES or DELETES. Insert only.

```
CREATE TABLE audit_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID REFERENCES tenants(id),              -- NULL for super
  admin actions
  user_id     UUID REFERENCES users(id),

  -- What happened
  action       VARCHAR(100) NOT NULL,
  -- TENANT_APPROVED | TENANT_REJECTED | TENANT_SUSPENDED
  -- | API_KEY_GENERATED | API_KEY_ROTATED
  -- | ALERT_INGESTED | ALERT_MASKING_COMPLETE |
```

```

ALERT_DRAFT_GENERATED
    -- | DRAFT_EDITED | ALERT_APPROVED | ALERT_REJECTED
    -- | SAR_REHYDRATED | PDF_GENERATED | WEBHOOK_DELIVERED |
WEBHOOK_FAILED
    -- | SCHEMA_UPDATED | WEBHOOK_CONFIG_UPDATED |
TEST_ALERT_SUBMITTED

-- Context
entity_type      VARCHAR(50),                                -- alert | sar_draft |
tenant | user
entity_id        UUID,

-- Details (arbitrary metadata about the action)
details          JSONB NOT NULL DEFAULT '{}',

-- Actor context
actor_ip         VARCHAR(50),
actor_user_agent VARCHAR(500),

created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
-- NO updated_at. This table is append-only.
);

CREATE INDEX idx_audit_tenant_id ON audit_logs(tenant_id, created_at DESC);
CREATE INDEX idx_audit_entity ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_audit_action ON audit_logs(action, created_at DESC);

```

13. api_logs

HTTP request/response log for every call to the ingestion and tenant APIs.

```

CREATE TABLE api_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID REFERENCES tenants(id),
  user_id     UUID REFERENCES users(id),

  method      VARCHAR(10) NOT NULL,
  endpoint    VARCHAR(255) NOT NULL,
  status_code INTEGER NOT NULL,

  request_ip  VARCHAR(50),
  user_agent  VARCHAR(500),

  -- Performance
  latency_ms  INTEGER,

  -- Error detail (only set on 4xx/5xx)
  error_code  VARCHAR(50),
  error_message TEXT,

```

```

        created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );

    CREATE INDEX idx_api_logs_tenant_id ON api_logs(tenant_id, created_at DESC);
    CREATE INDEX idx_api_logs_endpoint ON api_logs(endpoint, created_at DESC);
    CREATE INDEX idx_api_logs_status ON api_logs(status_code, created_at DESC);

    -- Partition by month in production (not required for MVP)

```

Seed Data

Super Admin User

```

INSERT INTO users (id, tenant_id, email, password_hash, full_name, role)
VALUES (
    gen_random_uuid(),
    NULL,
    'admin@aegis-aml.com',
    '$2b$12$<bcrypt hash of "AegisAdmin2026!">',
    'Aegis Super Admin',
    'SUPER_ADMIN'
);

```

Demo Tenant (Pre-approved for demo sessions)

```

INSERT INTO tenants (
    id, name, slug, company_type, status, api_key_hash, api_key_prefix,
    tenant_id_public,
    approved_at, created_at
) VALUES (
    gen_random_uuid(),
    'DemoFintech Pvt Ltd',
    'demofintech',
    'FINTECH',
    'ACTIVE',
    '<bcrypt of demo-key>',
    'sk-ae-demo',
    'TEN-DEMO',
    NOW(),
    NOW()
);

INSERT INTO ingestion_schemas (tenant_id, name, template_key, field_map,
pii_fields, is_active)
VALUES (
    (SELECT id FROM tenants WHERE slug = 'demofintech'),
    'Standard Fintech Transaction Alert',
    'STANDARD_FINTECH',

```

```

    '{
      "customer_name": "customer.full_name",
      "customer_id": "customer.id",
      "account_id": "account.number",
      "transaction_id": "txn.ref_id",
      "transaction_amount": "txn.amount",
      "transaction_currency": "txn.currency",
      "transaction_type": "txn.type",
      "transaction_direction": "txn.direction",
      "transaction_timestamp": "txn.timestamp",
      "counterparty_account": "counterparty.account",
      "counterparty_name": "counterparty.name",
      "counterparty_institution": "counterparty.bank",
      "ip_address": "metadata.ip",
      "device_id": "metadata.device_id",
      "risk_score": "risk.score",
      "alert_reason": "risk.reason"
    }',

    '["customer_name","customer_id","account_id","counterparty_account","counterparty_
name","ip_address","device_id"]',
    TRUE
);

INSERT INTO webhook_configs (tenant_id, use_internal_sink, is_active)
VALUES (
  (SELECT id FROM tenants WHERE slug = 'demofintech'),
  TRUE,
  TRUE
);

INSERT INTO llm_configs (tenant_id, provider, model_name, sar_template_style)
VALUES (
  (SELECT id FROM tenants WHERE slug = 'demofintech'),
  'GROQ',
  'llama-3.3-70b-versatile',
  'BOTH'
);

```

Preset Schema Templates (Hardcoded in Backend)

Stored as Python constants in `app/data/schema_presets.py` (not in DB — loaded at startup):

```

SCHEMA_PRESETS = {
  "STANDARD_FINTECH": {
    "name": "Standard Fintech Transaction Alert",
    "field_map": {
      "customer_name": "customer.full_name",
      "customer_id": "customer.id",
      "account_id": "account.number",
      ...
    }
  }
}

```

```

    },
    "pii_fields": ["customer_name", "customer_id", "account_id", ...]
  },
  "SEBI_BROKER": {
    "name": "SEBI Stock Broker Trading Alert",
    "field_map": {
      "customer_name": "client.name",
      "customer_id": "client.pan",          # PAN number for SEBI
      "account_id": "demat.account_id",
      "transaction_amount": "trade.value",
      ...
    },
    "pii_fields": ["customer_name", "customer_id", "account_id", ...]
  },
  "PAYMENT_GW": {
    "name": "Payment Gateway Alert",
    "field_map": {
      "customer_name": "payer.name",
      "customer_id": "payer.vpa",          # UPI VPA
      "account_id": "payer.account",
      ...
    },
    "pii_fields": ["customer_name", "customer_id", "account_id", ...]
  }
}

```

Migration Strategy

Using Alembic with the following approach:

1. `alembic init alembic` in project root
2. Each table is a separate migration file for clean rollbacks
3. Migration order follows FK dependency order (tenants → users → schemas → alerts → ...)
4. Seed data applied via a separate `seed.py` script, not a migration